

IMC round 1

```
import pandas as pd
import numpy as np
import os
import sys
```

```
df1 =
    ↪ pd.read_csv('round-1-island-data-bottle/round-1-island-data-bottle/prices_round_1_day_-2')
df2 =
    ↪ pd.read_csv('round-1-island-data-bottle/round-1-island-data-bottle/prices_round_1_day_-1')
df3 =
    ↪ pd.read_csv('round-1-island-data-bottle/round-1-island-data-bottle/prices_round_1_day_0')
```

```
df_squid = pd.concat([df1[df1['product'] == 'SQUID_INK'],df2[df2['product']
    ↪ == 'SQUID_INK'],df3[df3['product'] ==
    ↪ 'SQUID_INK']]).reset_index(drop=True)
df_kelp = pd.concat([df1[df1['product'] == 'KELP'],df2[df2['product'] ==
    ↪ 'KELP'],df3[df3['product'] == 'KELP']]).reset_index(drop=True)
```

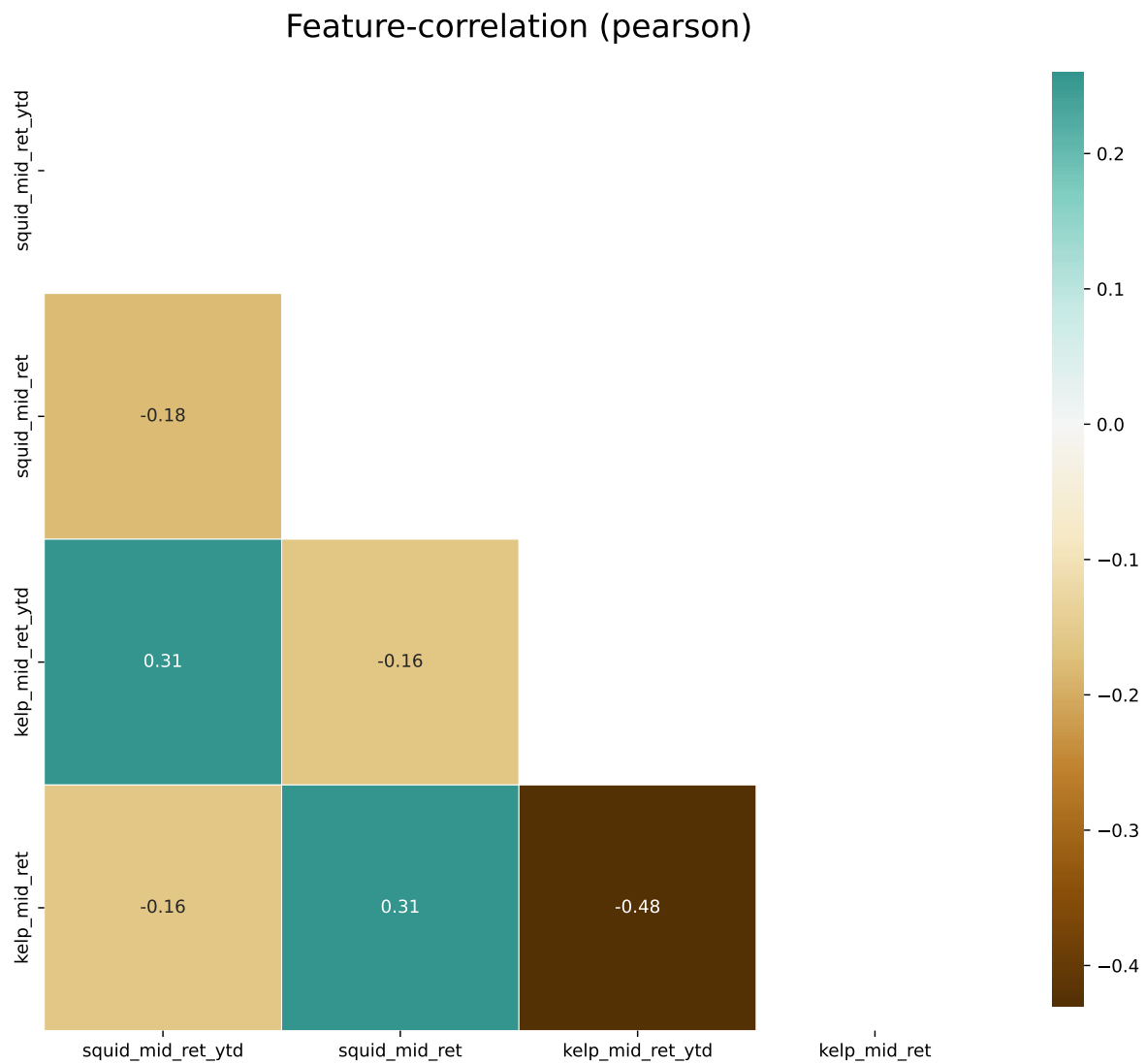
```
df_m =
    ↪ pd.merge(df_squid[['day','timestamp','mid_price']].rename(columns={'mid_price':'squid_mid_price'}),df_kelp[['day','timestamp','mid_price']].rename(columns={'mid_price':'kelp_mid_price'}))
```

```
df_m['squid_mid_price_ytd'] = df_m['squid_mid_price'].shift(1)
df_m['kelp_mid_price_ytd'] = df_m['kelp_mid_price'].shift(1)
```

```
df_m['squid_mid_ret'] = df_m['squid_mid_price'].pct_change()
df_m['kelp_mid_ret'] = df_m['kelp_mid_price'].pct_change()
df_m['squid_mid_ret_ytd'] = df_m['squid_mid_ret'].shift(1)
df_m['kelp_mid_ret_ytd'] = df_m['kelp_mid_ret'].shift(1)
```

```
import klib as kl
```

```
kl.corr_plot(df_m[['squid_mid_ret_ytd','squid_mid_ret','kelp_mid_ret_ytd','kelp_mid_ret']])
```



```
df_m.set_index(['day','timestamp'], inplace=True)
```

```
import pandas as pd  
import numpy as np
```

```

import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
import statsmodels.api as sm
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.stats.diagnostic import lilliefors
import warnings
warnings.filterwarnings('ignore')

```

```

# Set the style for better visualizations
plt.style.use('seaborn-v0_8-whitegrid')
sns.set_palette("deep")
plt.rcParams['figure.figsize'] = [12, 8]

```

```

for col in ['squid_mid_ret', 'kelp_mid_ret']:
    # Basic descriptive statistics
    stats_df = pd.DataFrame({
        'Statistic': [
            'Count', 'Mean', 'Median', 'Std Dev', 'Skewness', 'Kurtosis',
            'Min', 'Max', '25%', '75%', 'Abs Mean', 'Positive Returns %'
        ],
        'Value': [
            df_m[col].count(),
            df_m[col].mean(),
            df_m[col].median(),
            df_m[col].std(),
            df_m[col].skew(),
            df_m[col].kurt(),
            df_m[col].min(),
            df_m[col].max(),
            df_m[col].quantile(0.25),
            df_m[col].quantile(0.75),
            df_m[col].abs().mean(),
            (df_m[col] > 0).mean() * 100
        ]
    })

    print(stats_df)

# Reset index to plot with timestamp
df_plot = df_m.copy().reset_index()

```

```

df_plot['index'] = ((df_plot['day']+3)*1000000) + df_plot['timestamp']

# Plotting the returns over time
plt.figure(figsize=(14, 7))
plt.plot(df_plot['index'], df_plot[col], linewidth=1.5)
plt.title(f'{col} Returns Over Time', fontsize=16)
plt.axhline(y=0, color='r', linestyle='-', alpha=0.3)
plt.xlabel('Timestamp')
plt.ylabel('Returns')
plt.grid(True, alpha=0.3)
plt.show()

# Plot by day

df_plot2 = df_plot.reset_index()

plt.figure(figsize=(14, 10))
for day in sorted(df_plot2['day'].unique()):
    day_data = df_plot2[df_plot2['day'] == day]
    plt.subplot(len(df_plot2['day'].unique()), 1, int(day) + 3) # +3
    ↪ because days start at -2
    plt.plot(day_data['timestamp'], day_data[col], label=f'Day {day}')
    plt.axhline(y=0, color='r', linestyle='-', alpha=0.3)
    plt.title(f'Day {day}')
    plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

# Histogram and KDE
plt.figure(figsize=(14, 6))

plt.subplot(1, 2, 1)
sns.histplot(df_m[col].dropna(), kde=True, bins=30)
plt.axvline(x=0, color='r', linestyle='--', alpha=0.7)
plt.title(f'Distribution of {col} Returns', fontsize=14)
plt.xlabel('Returns')

plt.subplot(1, 2, 2)
sns.boxplot(y=df_m[col].dropna())
plt.title(f'Box Plot of {col} Returns', fontsize=14)
plt.ylabel('Returns')

```

```

plt.tight_layout()
plt.show()

# QQ Plot to check for normality
plt.figure(figsize=(10, 6))
sm.qqplot(df_m[col].dropna(), line='s')
plt.title(f'QQ Plot of {col} Returns', fontsize=14)
plt.grid(True, alpha=0.3)
plt.show()

# Run normality tests
shapiro_test = stats.shapiro(df_m[col].dropna())
ks_test = lilliefors(df_m[col].dropna())
ad_test = stats.anderson(df_m[col].dropna(), dist='norm')

print(f"Normality Tests for {col} Returns:")
print(f"Shapiro-Wilk Test: statistic={shapiro_test[0]:.4f},
      ↪ p-value={shapiro_test[1]:.4f}")
print(f"Kolmogorov-Smirnov Test: statistic={ks_test[0]:.4f},
      ↪ p-value={ks_test[1]:.4f}")
print("Anderson-Darling Test:")
for i in range(len(ad_test.critical_values)):
    sl, cv = ad_test.significance_level[i], ad_test.critical_values[i]
    print(f"      {sl}%: {cv:.3f}, {'Reject' if ad_test.statistic > cv else
          ↪ 'Accept'} null hypothesis")

# Autocorrelation analysis
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))
plot_acf(df_m[col].dropna(), lags=20, ax=ax1)
ax1.set_title('Autocorrelation Function (ACF)', fontsize=14)

plot_pacf(df_m[col].dropna(), lags=20, ax=ax2)
ax2.set_title('Partial Autocorrelation Function (PACF)', fontsize=14)

plt.tight_layout()
plt.show()

# Calculate rolling statistics
window_sizes = [5, 10, 20]
df_rolling = df_m.reset_index()

plt.figure(figsize=(14, 10))

```

```

# Rolling mean
plt.subplot(3, 1, 1)
plt.plot(df_rolling['timestamp'], df_rolling[col], label='Returns',
↪ alpha=0.5)
for window in window_sizes:
    plt.plot(df_rolling['timestamp'],
              df_rolling[col].rolling(window=window).mean(),
              label=f'{window}-period MA')
plt.title('Rolling Mean of SQUID_INK Returns', fontsize=14)
plt.legend()
plt.grid(True, alpha=0.3)

# Rolling volatility (standard deviation)
plt.subplot(3, 1, 2)
for window in window_sizes:
    plt.plot(df_rolling['timestamp'],
              df_rolling[col].rolling(window=window).std(),
              label=f'{window}-period SD')
plt.title('Rolling Volatility of SQUID_INK Returns', fontsize=14)
plt.legend()
plt.grid(True, alpha=0.3)

# Rolling min/max
plt.subplot(3, 1, 3)
for window in window_sizes:
    plt.plot(df_rolling['timestamp'],
              df_rolling[col].rolling(window=window).min(),
              label=f'{window}-period Min', linestyle='--')
    plt.plot(df_rolling['timestamp'],
              df_rolling[col].rolling(window=window).max(),
              label=f'{window}-period Max', linestyle='-.')
plt.title('Rolling Min/Max of SQUID_INK Returns', fontsize=14)
plt.legend()
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# Volatility clustering - plot squared returns
plt.figure(figsize=(14, 6))
plt.plot(df_rolling['timestamp'], df_rolling[col]**2)

```

```

plt.title('Squared Returns (Volatility Clustering)', fontsize=14)
plt.xlabel('Timestamp')
plt.ylabel('Squared Returns')
plt.grid(True, alpha=0.3)
plt.show()

# ACF of absolute returns to check volatility persistence
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))
plot_acf(df_m[col].dropna().abs(), lags=20, ax=ax1)
ax1.set_title('ACF of Absolute Returns', fontsize=14)

plot_acf(df_m[col].dropna()2, lags=20, ax=ax2)
ax2.set_title('ACF of Squared Returns', fontsize=14)

plt.tight_layout()
plt.show()

# Correlation with Kelp returns
plt.figure(figsize=(14, 6))
plt.scatter(df_m['kelp_mid_ret'], df_m[col], alpha=0.6)
plt.title('SQUID_INK Returns vs KELP Returns', fontsize=14)
plt.xlabel('KELP Returns')
plt.ylabel('SQUID_INK Returns')
plt.axhline(y=0, color='r', linestyle='--', alpha=0.3)
plt.axvline(x=0, color='r', linestyle='--', alpha=0.3)
plt.grid(True, alpha=0.3)

# Add correlation coefficient to the plot
corr = df_m[[col, 'kelp_mid_ret']].corr().iloc[0, 1]
plt.annotate(f'Correlation: {corr:.4f}',
             xy=(0.05, 0.95), xycoords='axes fraction',
             bbox=dict(boxstyle="round,pad=0.3", fc="white", ec="b",
↪ lw=1))
plt.show()

# Correlation over time
window = 10
df_rolling['rolling_corr'] =
↪ df_rolling[col].rolling(window).corr(df_rolling['kelp_mid_ret'])

plt.figure(figsize=(14, 6))
plt.plot(df_rolling['timestamp'], df_rolling['rolling_corr'])

```

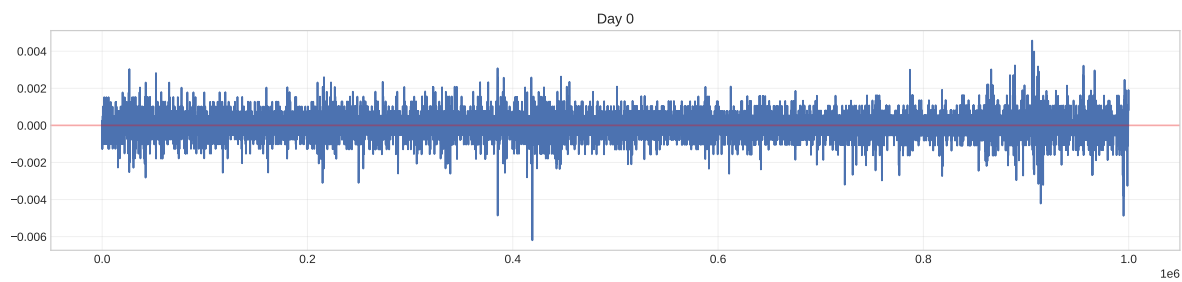
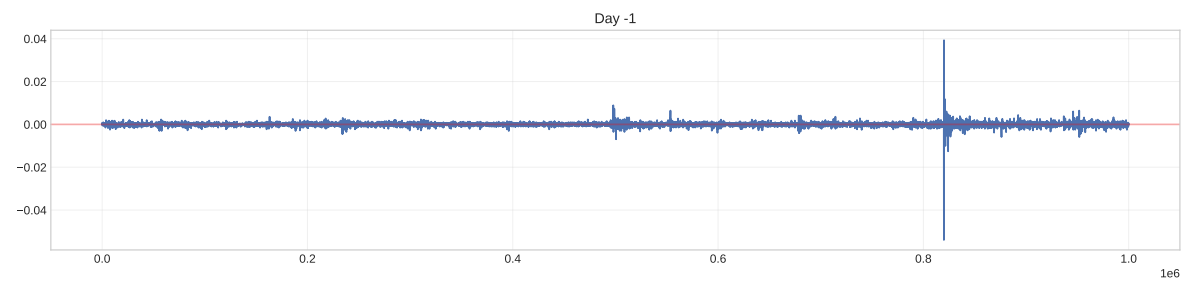
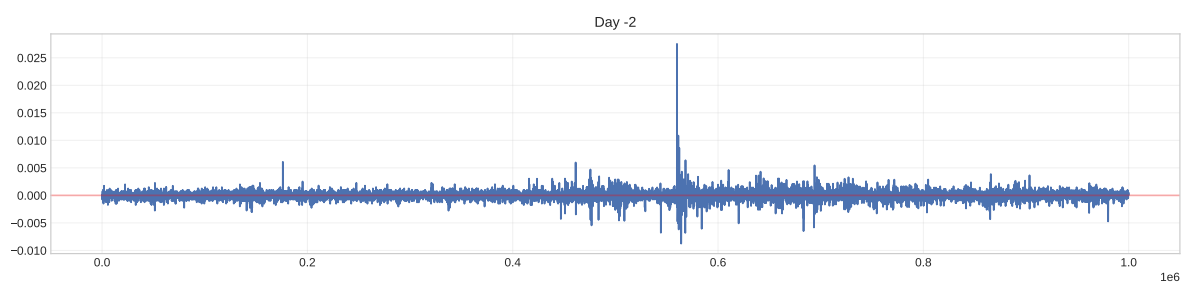
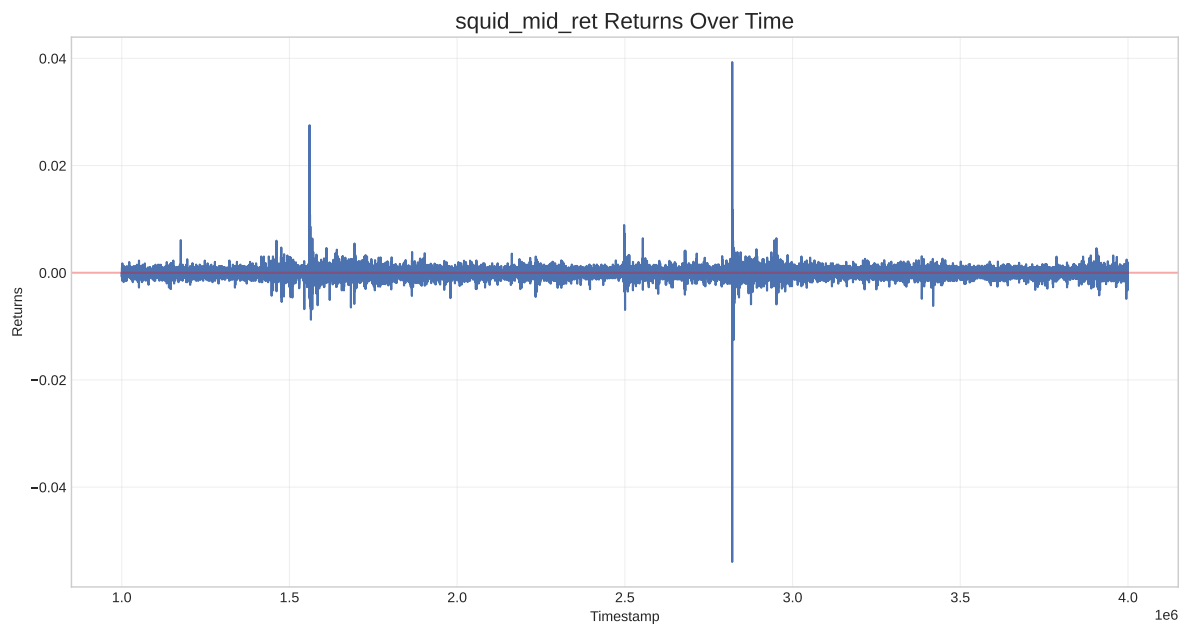
```

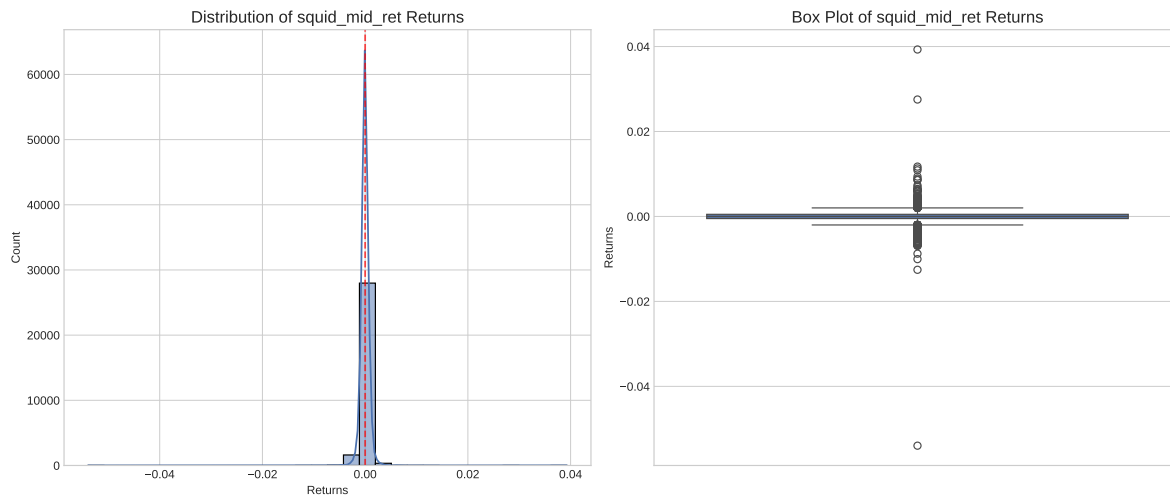
plt.title(f'Rolling {window}-period Correlation between SQUID_INK and
→ KELP Returns', fontsize=14)
plt.xlabel('Timestamp')
plt.ylabel('Correlation')
plt.axhline(y=0, color='r', linestyle='--', alpha=0.3)
plt.grid(True, alpha=0.3)
plt.show()

# Compare return distributions by day
plt.figure(figsize=(14, 6))
sns.violinplot(x='day', y=col, data=df_rolling)
plt.title('Distribution of SQUID_INK Returns by Day', fontsize=14)
plt.xlabel('Day')
plt.ylabel('Returns')
plt.grid(True, alpha=0.3)
plt.show()

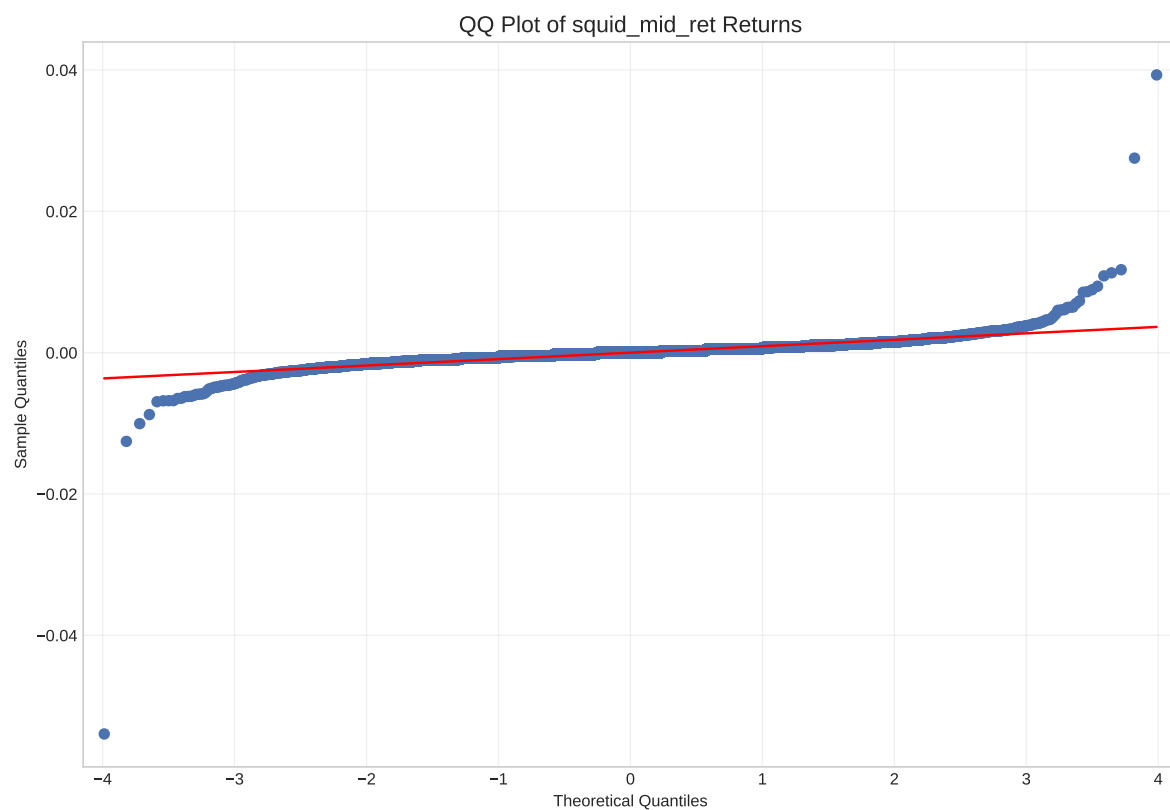
```

	Statistic	Value
0	Count	29999.000000
1	Mean	-0.000002
2	Median	0.000000
3	Std Dev	0.000912
4	Skewness	-3.308303
5	Kurtosis	559.090015
6	Min	-0.053963
7	Max	0.039301
8	25%	-0.000502
9	75%	0.000503
10	Abs Mean	0.000563
11	Positive Returns %	40.783333





<Figure size 3000x1800 with 0 Axes>



Normality Tests for squid_mid_ret Returns:

Shapiro-Wilk Test: statistic=0.7528, p-value=0.0000

Kolmogorov-Smirnov Test: statistic=0.1146, p-value=0.0010

Anderson-Darling Test:

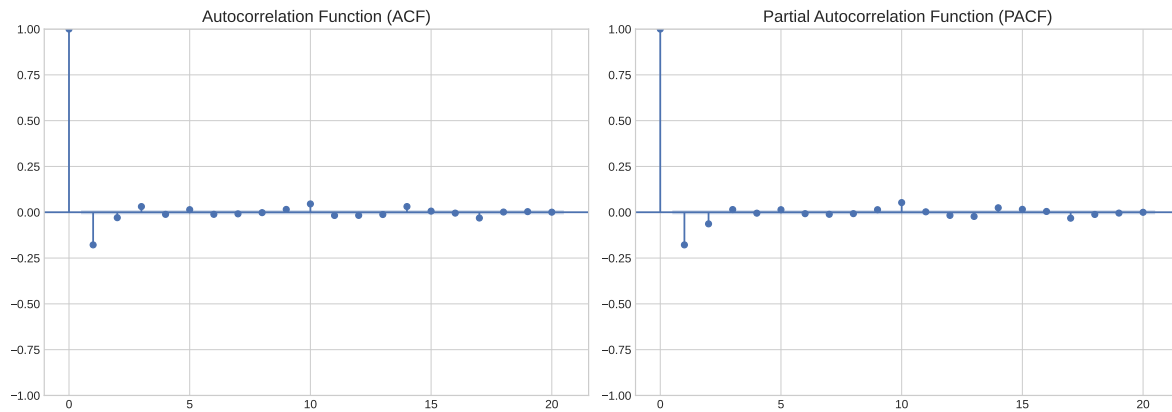
15.0%: 0.576, Reject null hypothesis

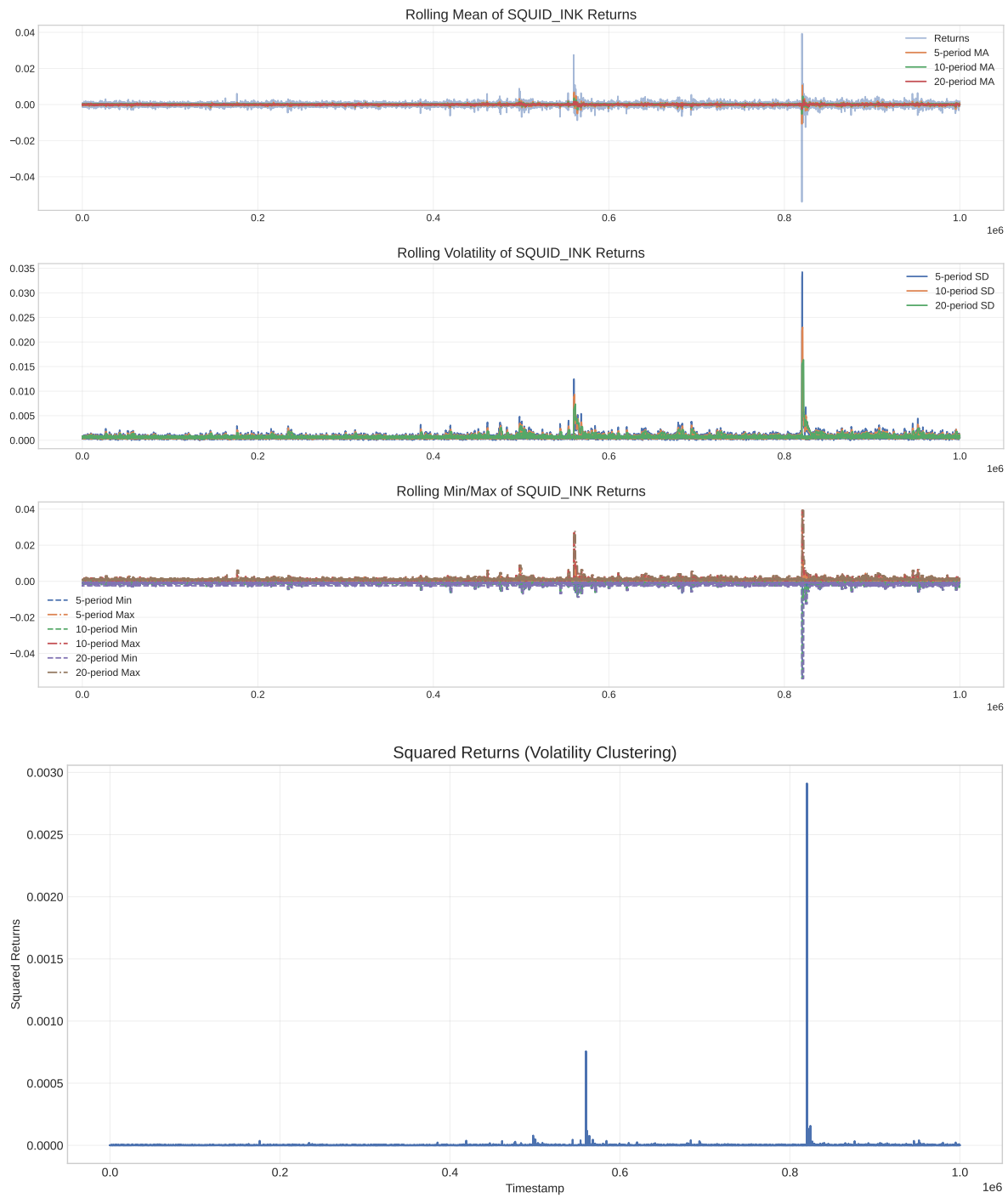
10.0%: 0.656, Reject null hypothesis

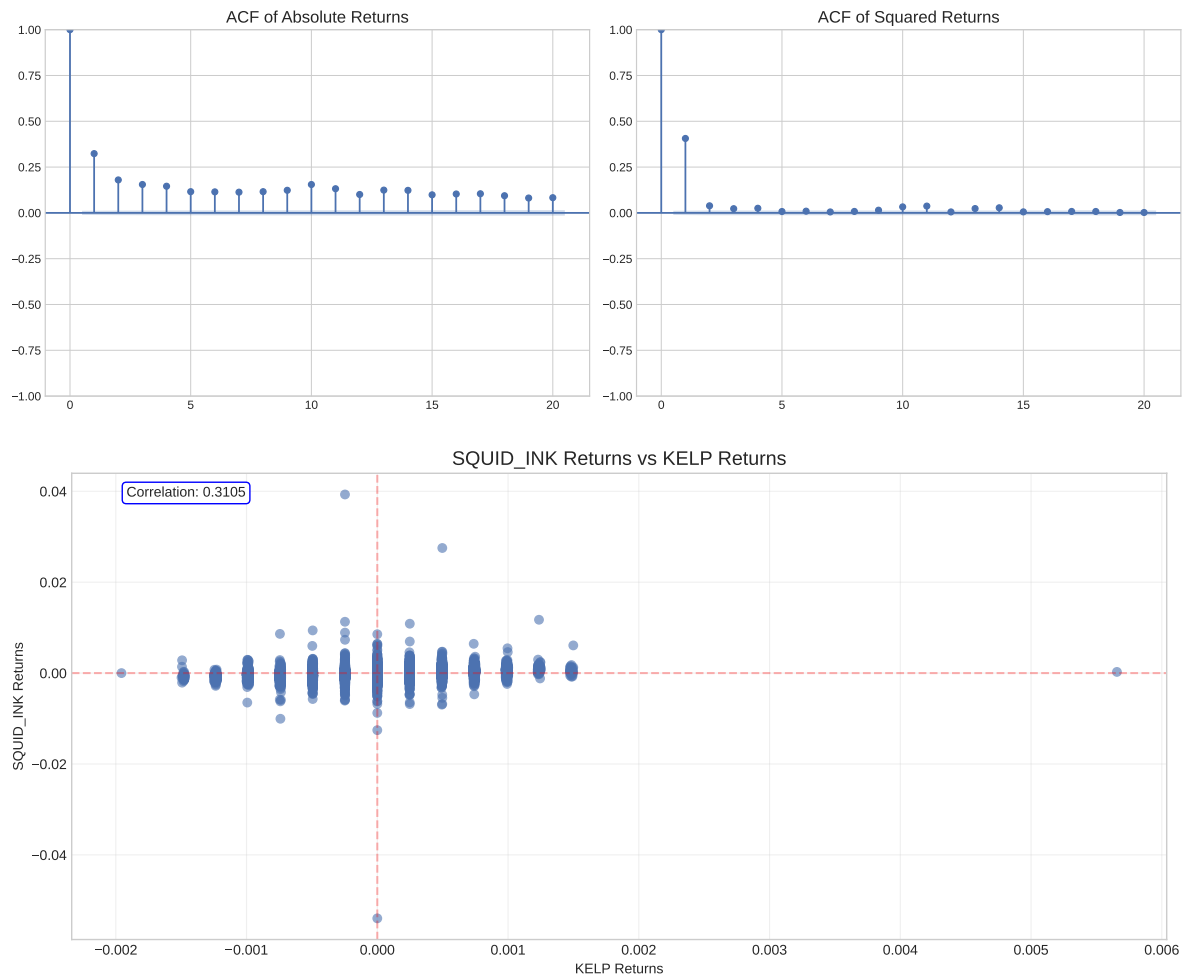
5.0%: 0.787, Reject null hypothesis

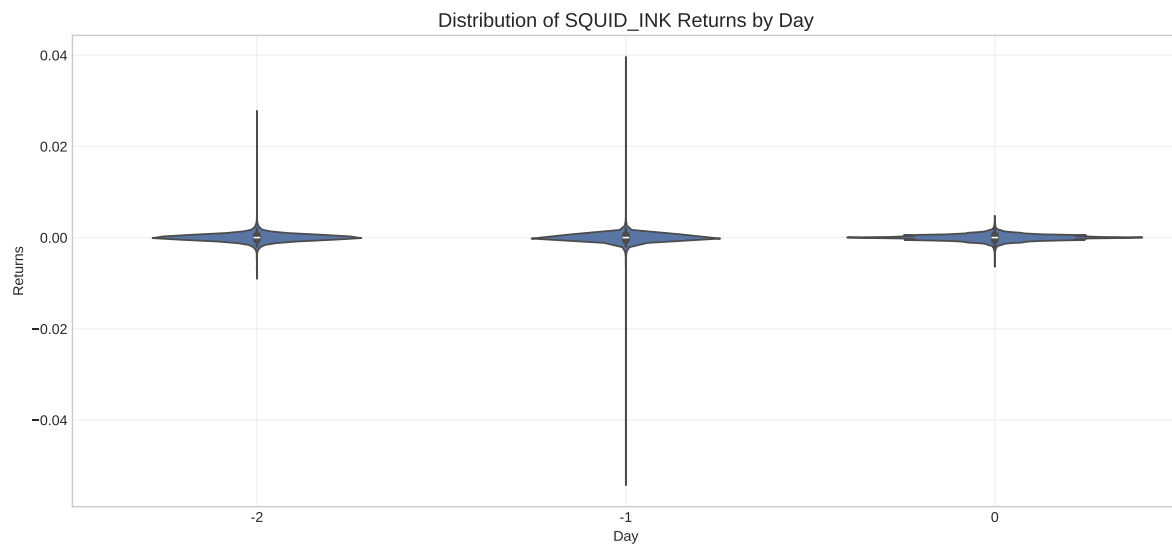
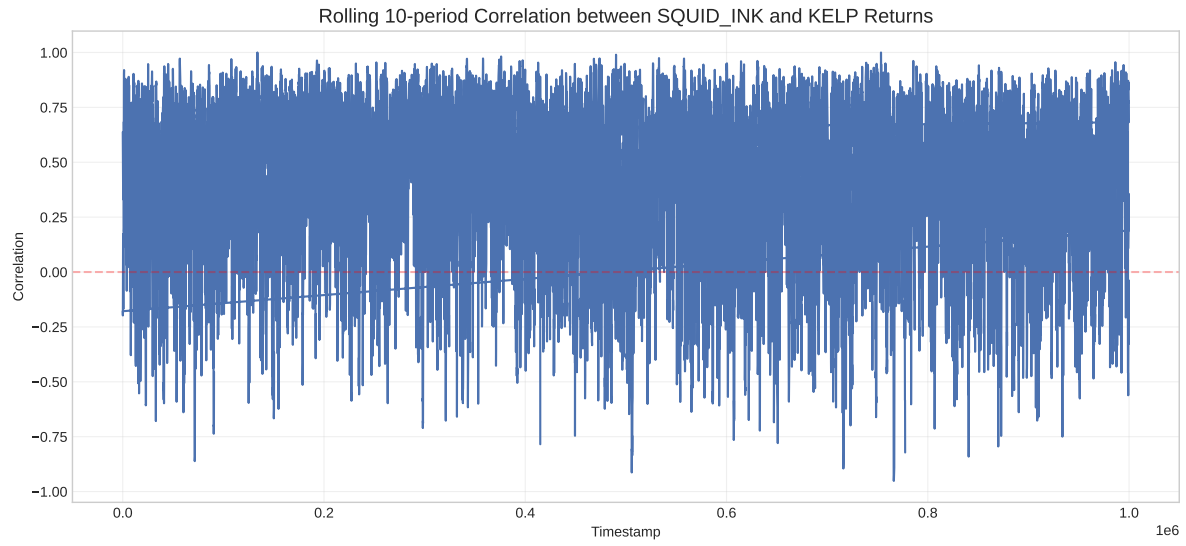
2.5%: 0.918, Reject null hypothesis

1.0%: 1.092, Reject null hypothesis



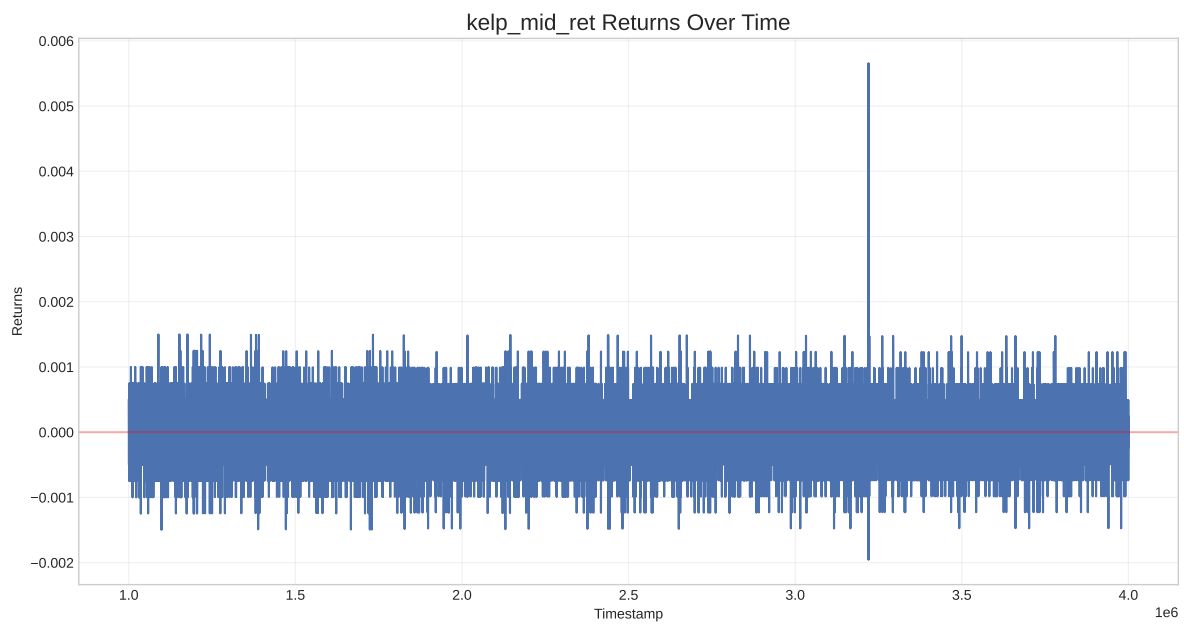


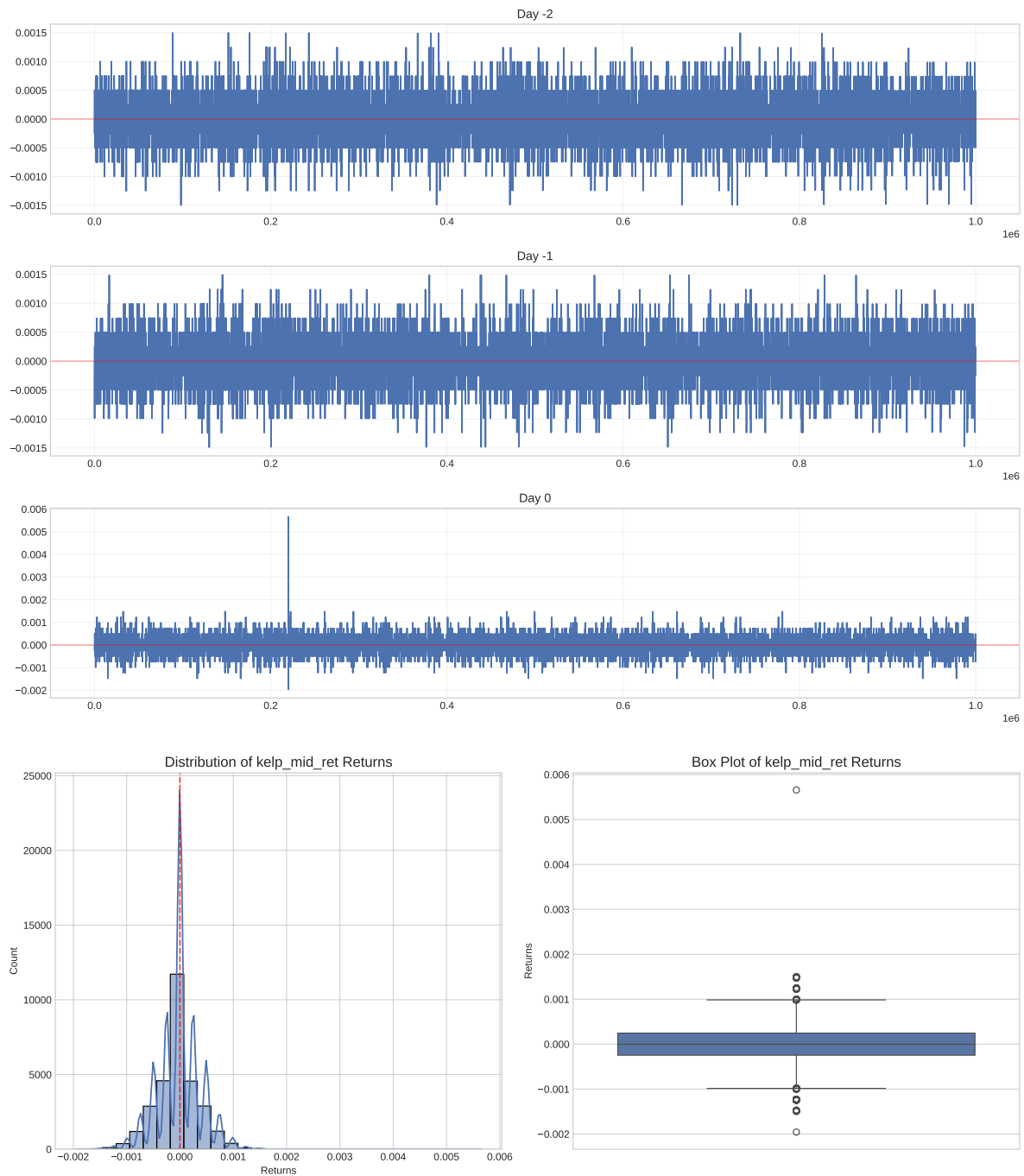




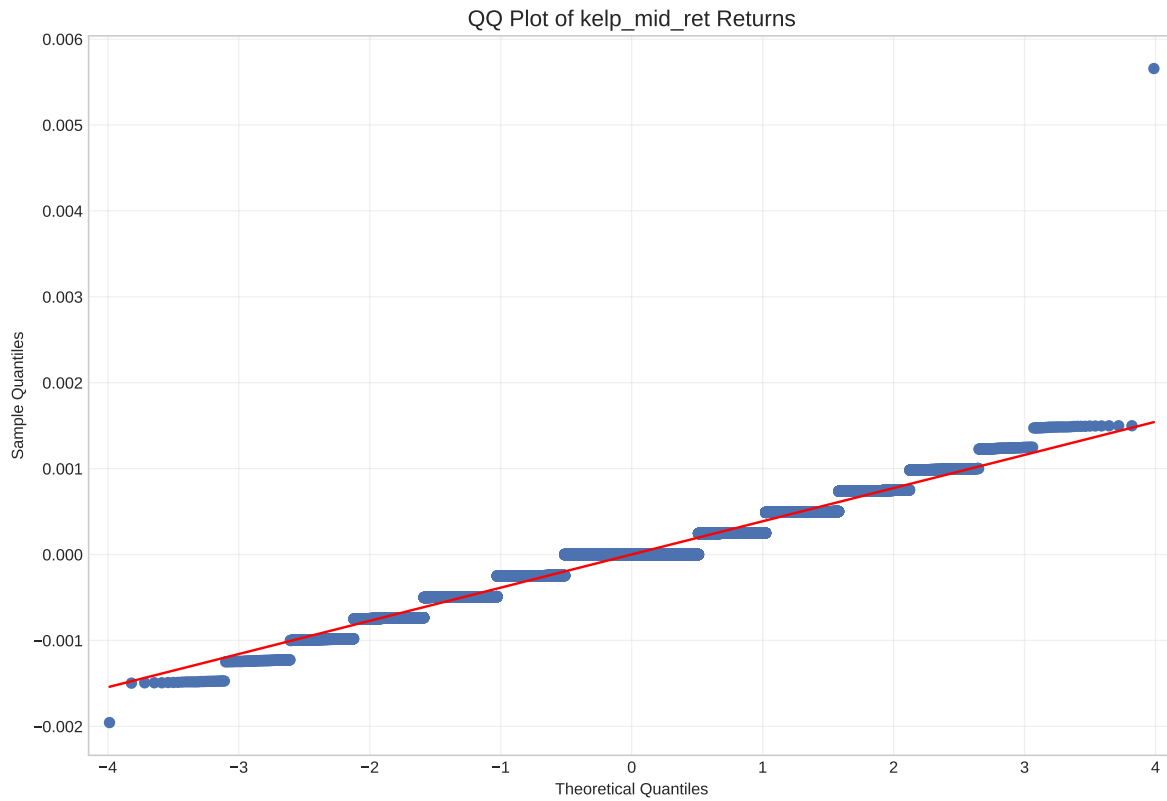
	Statistic	Value
0	Count	2.999900e+04
1	Mean	6.283120e-07
2	Median	0.000000e+00
3	Std Dev	3.862354e-04
4	Skewness	9.796893e-02
5	Kurtosis	2.386128e+00
6	Min	-1.956469e-03
7	Max	5.656665e-03
8	25%	-2.461841e-04

9	75%	2.463054×10^{-4}
10	Abs Mean	2.650698×10^{-4}
11	Positive Returns %	$3.049333 \times 10^{+01}$





<Figure size 3000x1800 with 0 Axes>



Normality Tests for kelp_mid_ret Returns:

Shapiro-Wilk Test: statistic=0.9440, p-value=0.0000

Kolmogorov-Smirnov Test: statistic=0.1957, p-value=0.0010

Anderson-Darling Test:

15.0%: 0.576, Reject null hypothesis

10.0%: 0.656, Reject null hypothesis

5.0%: 0.787, Reject null hypothesis

2.5%: 0.918, Reject null hypothesis

1.0%: 1.092, Reject null hypothesis

